

SOFTWARE CORRECTIVE, PREVENTIVE AND IMPROVEMENT ACTIONS PROCEDURE

RENTAMOVIL

**Thiago Fabian Rojas Guevara — David Santiago Erez Rodríguez — Nicole Dayana
Ramírez Vargas**

ANÁLISIS Y DESARROLLO DE SOFTWARE 3145556

SERVICIO NACIONAL DE APRENDIZAJE

2026

Reference Framework:

This procedure adopts the terminology and problem management cycle of the ISO/IEC 29110 standard (basic profile for Very Small Entities — VSE), without constituting a formal certification. It is used as a technical reference framework, adapted to the actual scope of the project.

1. CHANGE AND VERSION CONTROL

Version	Date	Author	Change Description
1.0	2026-10-09	David Erez	Initial version of the procedure: CAPA cycle, registration templates, and real findings identified during the comprehensive technical review of the project.

2. OBJECTIVE AND SCOPE

2.1 Objective

Establish a formal and repeatable procedure to identify, register, prioritize, correct, and prevent findings (defects, incidents, and improvement opportunities) of the RentaMóvil software throughout its life cycle, ensuring that each action is documented, has an assigned owner, and an effectiveness verification.

2.2 Scope

This procedure applies to the entire life cycle of the RentaMóvil web and mobile application, including:

- The backend microservices (rtm-iam, rtm-fleet-maintenance, rtm-booking-reservation, rtm-payment-billing, rtm-api-gateway) and their database migration repositories.
- The web front-end (React + Vite) and the mobile application (Expo + React Native).
- The execution infrastructure (Docker containers, shared SQL Server database).
- It does not replace the operational incident management process already documented in `rtm-docs/13-operations/incident-management.md`; it complements it, focusing on the registration and tracking of corrective/preventive/improvement actions derived from findings, rather than the live response during an incident.

3. CAPA CYCLE PROCEDURE

3.1 How a Finding is Reported

A finding can originate through any of these channels:

- **User or tester report:** errors observed while using the web or mobile app (e.g., console captures, reproduced error messages).
- **Monitoring alerts or logs:** errors captured in the backend service logs or in the API Gateway healthcheck.
- **Technical review / code review:** inconsistencies found when auditing code or documentation (documentation-vs-implementation gaps, technical debt).
- **Production incident:** any event classified and closed under the [rtm-docs/13-operations/incident-management.md](#) runbook that requires a follow-up action.

Every finding is recorded using the template in section 4, assigning it a consecutive number (C-xx for corrective, P-xx for preventive, M-xx for improvements).

3.2 How it is Evaluated and Prioritized

Each finding is classified according to its actual impact on system operations:

Priority	Criteria	Target Response Time	Real Project Example
Critical	Blocks a key business operation for all users (login, reservations, payments) or compromises security.	Immediate (same day)	Infinite login loop when making a reservation (unpersisted refresh token rotation)
High	Affects an important functionality but a workaround exists, or affects a subset of users.	1–2 business days	Mobile catalog showing unavailable vehicles (409 when making a reservation)
Medium	Affects the user experience without blocking operations (UX, missing data, minor inconsistencies).	Within the current sprint	Bank account QR code hidden in the UI
Low	Improvement or technical debt that does not affect current operations.	Backlog / next sprint	Complete risks.md, add CI/CD

3.3 CAPA Cycle Flow

Detection → Registration → Classification (priority) → Root Cause Analysis → Definition of Action (corrective, preventive, or improvement) → Execution → Effectiveness Verification → Closure.

A corrective action responds to a finding that has already occurred (eliminating the cause of a real problem). A preventive action responds to an identified risk before it materializes (eliminating the cause of a potential problem).

An improvement action does not correct or prevent a failure, but rather increases the quality, robustness, or maintainability of the software without an underlying defect.

4. RECORD FORMATS (BLANK TEMPLATES)

These templates are completed each time the team detects a new finding. One row/form per finding.

4.1 Template — Corrective Action

Record No.	Detection Date	Description of Finding	Type / Priority
Identified root cause:			
Proposed corrective action:			
Responsible Owner:			
Commitment date:			
Actual closure date:			
Effectiveness verification (how it was proven to work):			
Status (Open / In progress / Closed):			

4.2 Template — Acción Preventiva

Record No.	Detection Date	Description of Finding	Type / Priority
Identified root cause:			
Proposed preventive action:			
Responsible Owner:			
Commitment date:			
Actual closure date:			

Effectiveness verification (how it was proven to work):	
Status (Open / In progress / Closed):	

4.3 Template — Improvement Action

Record No.	Detection Date	Description of Finding	Type / Priority
Identified root cause:			
Proposed improvement action:			
Responsible Owner:			
Commitment date:			
Actual closure date:			
Effectiveness verification (how it was proven to work):			
Status (Open / In progress / Closed):			

5. REAL REGISTRATION — IDENTIFIED FINDINGS (BASELINE)

These are the actions already identified and, for the most part, already executed during the comprehensive technical review of the project (October 2026). They serve as the baseline of the procedure and as an example of how to fill out the templates in section 4. The full detail with file/line for each finding is in 000-notes/repaso-evaluacion-rentamovil/21-BUGS-E-INCONGRUENCIAS-ENCONTRADAS.md.

5.1 Corrective Actions

No.	Finding	Root Cause	Executed Corrective Action	Status
C-01	Login returned 400 on the mobile app.	The client sent {email, password}; rtm-iam requires {identifier, password}.	The payload in authService.ts was corrected to use identifier.	Closed

No.	Finding	Root Cause	Executed Corrective Action	Status
C-02	Registration returned 400 on the mobile app.	The client sent fields in snake_case (first_name); the backend expects camelCase (firstName).	Field mapping was adjusted in authService.ts.	Closed
C-03	Profile query failed (non-existent route).	The client called /auth/me, which was stripped to /me by the /auth prefix stripping; the actual route is /users/me.	Routing in env.ts (me and changePassword) was corrected to bypass the /auth strip.	Closed
C-04	Infinite 'return to login' loop when creating a reservation.	The refresh token rotates on each use (single-use); the client did not persist the new token received, causing reuse of an already consumed token and revocation of all sessions.	Corrected httpClient.ts and AuthContext.tsx to persist the rotated refreshToken after each renewal.	Closed
C-05	Reservations repeatedly failing with 409 'vehicle unavailable'.	The mobile catalog never implemented the availability filter by branch/dates (it did exist in the web front-end).	Added branchId to Vehicle, connected getAvailability(), and filtered in runSearch (HomePage.tsx).	Closed
C-06	Payment, reservation, and insurance routes with wrong names.	Misalignment between routes assumed by the client and real routes exposed by each microservice.	Corrected routes in paymentService.ts, reservationService.ts, and insuranceService.ts.	Closed
C-07	Client mappers assumed non-existent fields in the actual domain (accountNumber, tag, etc.).	Client DTOs were built before verifying the actual backend contract.	Rewrote bankAccountMapper.ts and insuranceMapper.ts against the real domain.	Closed
C-08	The bank account QR code was not displayed on the mobile app.	The rendering of the QR <Image> was commented out in BankAccountSelector.tsx.	Reactivated QR rendering, with a notice that current images are for testing purposes.	Closed
C-09	liquibase changelogSync marked 4 changesets as executed without executing them (rtm-booking-db).	changelogSync was used to skip a single changeset blocked by an external FK, but it also marked 4 other unrelated changesets.	Removed the 4 fake rows from DATABASECHANGELOG_BOOKING and re-executed liquibase:update.	Closed

No.	Finding	Root Cause	Executed Corrective Action	Status
C-10	Every reservation operation returned 401 'token does not include personId'.	Outdated local copy of rtm-iam; the develop branch already included the personId claim in the JWT.	Updated local environment with git merge --ff-only origin/develop (included SMTP config and person.image_url migration).	Closed — local infrastructure responsibility, not product code

5.2 Preventive Actions

No.	Risk to Prevent	Preventive Action	Where it Applies
P-01	Repeating the C-09 error (changelogSync over-marking).	Run liquibase status and review changeset by changeset before using changelogSync; never apply in batch.	The 4 *-db repos (iam, fleet, booking, payment)
P-02	Repeating the refresh token rotation bug (C-04) in new clients.	Explicitly document in the client integration guide that the refresh token is single-use and must be persisted after each renewal.	Integration guides for new clients (web, mobile, future)
P-03	Outdated local environments hiding real backend bugs (like C-10).	Synchronize each local checkout with origin/develop before starting an integration testing session.	Entire team's workflow
P-04	New client endpoints with malformed payloads (like C-01/C-02).	Validate the exact shape of the payload (field names, camelCase) against the real backend contract before integrating it, not against assumptions.	Development of new features in web and mobile
P-05	Reservation race condition going unnoticed in production.	Add a concurrency test (two simultaneous reservations for the same vehicle/dates) to the test suite before any real deployment.	rtm-booking-reservation

5.3 Improvement Actions

No.	Improvement Opportunity (not a bug)	Executed / Proposed Improvement Action	Status
M-01	rtm-docs/15-project-control/risks.md had the matrix ready but with no real risks registered.	The registry was completed with 5 real active risks (race condition, zero CI/CD, SQL Server as single point of failure, mobile store rejection, documentation-vs-code drift) and 2 lessons learned from incidents that already occurred (changelogSync, JWT without personId).	Closed

No.	Improvement Opportunity (not a bug)	Executed / Proposed Improvement Action	Status
M-02	Zero CI/CD pipelines across the 11 project repositories.	Pending: implement at least a basic workflow (build + tests) in GitHub Actions per repository. Formally registered as risk R-002.	Pending
M-03	There is no real protection against double booking of the same vehicle/dates (race condition, see P-05).	Pending: add @Transactional and a unique constraint at the database level in booking.reservation. Formally registered as risk R-001.	Pending
M-04	rtm-fleet-maintenance-db did not have a Dockerfile or docker-compose.yml, unlike the rest of the *-db repos.	Added Dockerfile, docker-compose.yml, and .env.example following the same pattern as rtm-booking-db/rtm-payment-billing-db; updated the README with the "With Docker" section and startup order.	Closed
M-05	rtm-docs/10-devops/local-setup.md and environments.md were generic templates (Node/Postgres/Redis/K8s/Vault) with no relation to the actual stack.	Both documents were rewritten with the real stack (Java/Spring Boot, SQL Server, Docker Compose), the startup order of the 10 repos, actual environment variables, real incidents as a common issues section, and the honest state of CI/CD and the production environment.	Closed

6. ROLES AND RESPONSIBILITIES

- **Development team:** detects, registers, and executes corrective/preventive/improvement actions in their component.
- **Technical leader / reviewer:** validates priority classification and approves the closure of each action after verifying its effectiveness.
- **Entire team:** responsible for maintaining this log — it is not the responsibility of a single person to keep it updated.